Лабораторная работа №1

Работа со строками на языке C/C++

Цель работы – освоить основы многопоточного программирования в C++, изучить создание и управление потоками с использованием библиотеки Boost.Thread, а также исследовать применение примитивов атомарности std::atomic для обеспечения корректной работы потоков.

Задачи работы

- Освоение работы с потоками в C++ с использованием Boost.Thread.
- Изучение синхронизации потоков с помощью std::atomic и std::mutex.
- Реализация многопоточных вычислительных алгоритмов.
- Анализ эффективности различных методов синхронизации.
- Сравнение производительности между Boost.Thread и std::atomic/std::mutex.

Теоретическая часть

1. **Определение многопоточности**

Многопоточное программирование (multithreading) — это метод выполнения нескольких потоков в одном процессе. Потоки позволяют программам работать параллельно, разделяя вычислительные ресурсы процессора.

Применение многопоточности

- Параллельные вычисления – увеличение производительности путём распараллеливания задач
- Асинхронные операции – повышение отзывчивости программ за счёт выполнения фоновых задач
- Оптимизация загрузки CPU – использование многоядерных процессоров
- Многопользовательские приложения – работа с несколькими запросами одновременно (например, серверы)

2. **Основы потоков в C++**

C++ предоставляет std::thread для создания и управления потоками. Для расширенных возможностей часто используют Boost.Thread, который обеспечивает лучшую кроссплатформенность и дополнительные инструменты синхронизации.

2.1. Создание потоков

Создать поток можно несколькими способами

- **Использование std::thread**

```
#include <iostream>
#include <thread>
void task() {
   std::cout << "Поток выполняется\n";
}
int main() {
   std::thread t(task);
   t.join();  // Ожидание завершения потока
   return 0;
}
```

Метод join() заставляет основной поток ждать завершения t.

- **Использование Boost.Thread**

```
#include <boost/thread.hpp>
#include <iostream>
void task() {
   std::cout << "Поток выполняется\n";
}
int main() {
   boost::thread t(task);
   t.join();
   return 0;
}
```

Boost.Thread предоставляет аналогичный функционал, но с расширенными возможностями.

2.2. Передача аргументов в поток

Передавать аргументы в поток можно несколькими способами.

- **Передача копии значений**

```
#include <boost/thread.hpp>
#include <iostream>
void task(int a, int b) {
    std::cout << "Сумма: " << (a + b) << "\n";
}
int main() {
    boost::thread t(task, 5, 10);
    t.join();
    return 0;
}
```

Аргументы передаются по значению, что может быть неэффективно для больших объектов.

- **Передача по ссылке**

```
#include <boost/thread.hpp>
#include <iostream>
void task(int &x) {
    x += 10;
}
int main() {
    int value = 5;
    boost::thread t(task, std::ref(value));
    t.join();
    std::cout << "Значение: " << value << "\n";  // 15
    return 0;
}
```

Использование std::ref() позволяет передавать аргументы по ссылке.

- **Передача через std::move()**

Если передаваемый объект нельзя копировать, используем std::move()

```
#include <boost/thread.hpp>
#include <iostream>
#include <vector>
void task(std::vector<int> v) {
    std::cout << "Размер вектора: " << v.size() << "\n";
}
int main() {
    std::vector<int> data = {1, 2, 3, 4, 5};
    boost::thread t(task, std::move(data));
    t.join();
    return 0;
}
```

После передачи объекта с std::move(), он становится недоступным в вызывающем коде.

2.3. Создание нескольких потоков

При наличии многоядерного процессора можно создать несколько потоков

```
#include <boost/thread.hpp>
#include <vector>
void task(int id) {
    std::cout << "Поток " << id << " выполняется\n";
}
int main() {
    std::vector<boost::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back(task, i);
    }
```

```
    for (auto &t : threads) t.join();
    return 0;
}
```

3. Основные проблемы многопоточности

При работе с потоками возможны следующие проблемы:

- Гонки данных (data race) – несколько потоков изменяют одни и те же данные без синхронизации
- Доступ к критической секции – ситуация, когда несколько потоков могут одновременно изменять ресурс
- Взаимные блокировки (deadlocks) – два или более потоков ожидают освобождения ресурсов, заблокированных друг другом

3.1. Гонки данных (data race)

Гонка данных возникает, когда несколько потоков одновременно обращаются к одной переменной без синхронизации.

Пример гонки данных

```
#include <iostream>
#include <thread>
int counter = 0;
void increment() {
    for (int i = 0; i < 1000; ++i) {
        counter++;
    }
}
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "Counter: " << counter << std::endl;  // Результат непредсказуем
```

```
    return 0;
}
```

Решить проблему можно с помощью std::atomic или std::mutex.

- **std::mutex**

Используется для блокировки доступа к данным.

```
#include <iostream>
#include <thread>
#include <mutex>
std::mutex mtx;
int counter = 0;
void increment() {
    for (int i = 0; i < 1000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        counter++;
    }
}
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "Counter: " << counter << std::endl;
    return 0;
}
```

Метод std::lock_guard автоматически освобождает мьютекс, когда выходит из области видимости.

- **std::atomic**

Атомарные переменные не требуют явной блокировки.

```
#include <iostream>
#include <thread>
#include <atomic>
std::atomic<int> counter(0);
void increment() {
    for (int i = 0; i < 1000; ++i) {
        counter++;
    }
}
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "Counter: " << counter.load() << std::endl;
    return 0;
}
```

Разница между std::mutex и std::atomic:

Метод	**Применение**	**Производительность**
std::mutex	Подходит для сложных структур	Может замедлять работу
std::atomic	Оптимален для простых переменных	Быстрее

3.2. Взаимные блокировки (deadlocks)

Deadlock – это ситуация, когда два потока ждут освобождения ресурса, заблокированного другим потоком.

```
#include <iostream>
#include <thread>
```

```
#include <mutex>
std::mutex mtx1, mtx2;
void task1() {
    std::lock_guard<std::mutex> lock1(mtx1);
    std::lock_guard<std::mutex> lock2(mtx2);
    std::cout << "Задача 1 выполнена\n";
}
void task2() {
    std::lock_guard<std::mutex> lock2(mtx2);
    std::lock_guard<std::mutex> lock1(mtx1);
    std::cout << "Задача 2 выполнена\n";
}
int main() {
    std::thread t1(task1);
    std::thread t2(task2);
    t1.join();
    t2.join();
    return 0;
}
```

Решение: Использовать std::lock() для предотвращения взаимной блокировки.

```
void task1() {
    std::lock(mtx1, mtx2);
    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
}
```

4. Производительность многопоточности

Производительность зависит от:

- Числа потоков (обычно равно количеству ядер CPU).

- Используемой синхронизации (std::atomic быстрее std::mutex).
- Типа задачи (чем больше вычислений, тем лучше масштабирование).

Для тестирования можно использовать std::chrono:

```
#include <chrono>

auto start = std::chrono::high_resolution_clock::now();

// код, который выполняется

auto end = std::chrono::high_resolution_clock::now();

std::cout << "Время: " << std::chrono::duration<double>(end - start).count() << " секунд\n";
```

5. Дополнительные методы многопоточного программирования

5.1. std::this_thread::sleep_for и std::this_thread::sleep_until

Позволяют приостановить выполнение потока на заданный период времени.

- std::this_thread::sleep_for

Приостанавливает выполнение потока на указанное время.

```
#include <iostream>

#include <thread>

#include <chrono>

void task() {

    std::cout << "Начало работы потока...\n";

    std::this_thread::sleep_for(std::chrono::seconds(3));

    std::cout << "Поток завершил работу через 3 секунды\n";

}

int main() {

    std::thread t(task);

    t.join();

    return 0;

}
```

- std::this_thread::sleep_until

Приостанавливает выполнение потока до определённого момента времени.

```
#include <iostream>
#include <thread>
#include <chrono>
void task() {
    auto wake_up_time = std::chrono::system_clock::now() + std::chrono::seconds(5);
    std::cout << "Поток будет ждать 5 секунд...\n";
    std::this_thread::sleep_until(wake_up_time);
    std::cout << "Продолжение работы потока\n";
}
int main() {
    std::thread t(task);
    t.join();
    return 0;
}
```

Вариант 1

Задание

Задача 1: Многопоточное вычисление чисел Фибоначчи с Boost.Thread

- Написать программу, создающую N потоков с использованием Boost.Thread.
- Каждый поток должен вычислять M-е число Фибоначчи (например, 40-е) рекурсивным методом.
- Главный поток должен ожидать завершения всех потоков и измерять время выполнения всей программы.
- Сравнить время выполнения многопоточного вычисления с последовательным вычислением.

Задача 2: Использование std::atomic и std::mutex для конкурентного счётчика

- Написать программу, в которой два потока увеличивают общий счётчик на 1 миллион раз.
- Реализовать три варианта кода:
  1. Без синхронизации (с демонстрацией гонки данных).
  2. С использованием std::atomic<int>.
  3. С использованием std::mutex.
- Замерить и сравнить производительность каждого метода.

В каждой задаче необходимо

- Провести тестирование на разных количествах потоков (например, 2, 4, 8).
- Сравнить скорость работы Boost.Thread и std::atomic/std::mutex.
- Сделать вывод о том, какие методы синхронизации более эффективны в разных сценариях.

Вариант 2

Задание

Задача 1: Параллельный поиск простых чисел с Boost.Thread

- Реализовать многопоточный алгоритм поиска простых чисел в диапазоне [1, N], используя K потоков.
- Каждому потоку передаётся свой поддиапазон, который он проверяет на простые числа.
- Главный поток собирает результаты и выводит общее количество найденных простых чисел.
- Замерить и сравнить время выполнения программы в однопоточном и многопоточном режимах.

Задача 2: Конкурентное накопление суммы массива с std::atomic и std::mutex

- Написать программу, которая вычисляет сумму элементов большого массива (10^7 элементов).
- Разделить вычисление на N потоков.
- Реализовать три варианта кода:
  1. С std::atomic<long long>.
  2. С std::mutex.
  3. Без синхронизации (для демонстрации ошибки).
- Сравнить производительность каждого метода.

В каждой задаче необходимо

- Провести тестирование на разных количествах потоков (например, 2, 4, 8).
- Сравнить скорость работы Boost.Thread и std::atomic/std::mutex.
- Сделать вывод о том, какие методы синхронизации более эффективны в разных сценариях.

Вариант 3

Задание

Задача 1: Умножение матриц с Boost.Thread

- Написать программу для многопоточного умножения двух квадратных матриц N×N, используя Boost.Thread.
- Каждому потоку передаётся вычисление одной строки результирующей матрицы.
- Сравнить время выполнения программы при различных значениях N (например, N = 100, 500, 1000).
- Провести сравнение с однопоточным выполнением.

Задача 2: Контроль доступа к общему ресурсу с std::atomic и std::mutex

- Реализовать систему банковских счетов, где N клиентов одновременно совершают транзакции.
- Каждому потоку назначается клиент, который случайным образом увеличивает или уменьшает баланс.
- Реализовать три варианта защиты:
  1. Без синхронизации (с демонстрацией аномалий).
  2. С использованием std::atomic<int>.
  3. С использованием std::mutex.
- Проанализировать, какой метод даёт лучшие результаты по скорости и корректности.

В каждой задаче необходимо

- Провести тестирование на разных количествах потоков (например, 2, 4, 8).
- Сравнить скорость работы Boost.Thread и std::atomic/std::mutex.
- Сделать вывод о том, какие методы синхронизации более эффективны в разных сценариях.

Вариант 4

Задание

Задача 1: Параллельная сортировка массива с Boost.Thread

- Реализовать многопоточный алгоритм сортировки массива из 10^6 элементов, используя Boost.Thread.
- Разделить массив на N частей, отсортировать их в отдельных потоках, затем объединить результаты.
- Сравнить скорость работы с однопоточной сортировкой.

Задача 2: Безопасная работа с очередью задач с std::atomic и std::mutex

- Реализовать многопоточную очередь задач, в которой N потоков добавляют задачи, а M потоков их извлекают.
- Реализовать три варианта синхронизации:
  1. Без синхронизации (с демонстрацией проблем).
  2. С std::atomic для управления состоянием очереди.
  3. С std::mutex для блокировки доступа к данным.
- Замерить производительность каждого подхода.

В каждой задаче необходимо

- Провести тестирование на разных количествах потоков (например, 2, 4, 8).
- Сравнить скорость работы Boost.Thread и std::atomic/std::mutex.
- Сделать вывод о том, какие методы синхронизации более эффективны в разных сценариях.

Дополнительные материалы

std::chrono

std::chrono — это библиотека в C++, предназначенная для работы с временем.

Она предоставляет высокоуровневые абстракции для работы с временем, измерениями интервалов и таймерами.

Все классы, функции и типы времени в std::chrono находятся в пространстве имен std.

Основные компоненты std::chrono

1. **Типы времени**

- std::chrono::duration<T, Ratio>

Это шаблонный тип для представления продолжительности времени.

Он хранит количество тиков какого-либо временного интервала, где T — это тип для хранения числового значения времени (например, int, long), а Ratio — это отношение временной единицы (например, секунды, миллисекунды).

Пример

std::chrono::duration<int> d(10); // 10 секунд

- std::chrono::time_point<T>

Это шаблонный тип, представляющий момент времени, основанный на длительности с некоторого начального времени (например, эпохи UNIX).

Он включает в себя точку в системе времени, например, с начала эпохи или с начала программы.

Пример

auto now = std::chrono::steady_clock::now(); // Текущий момент времени

2. **Временные единицы**

std::chrono предоставляет различные типы времени, такие как:

- std::chrono::seconds
- std::chrono::milliseconds
- std::chrono::microseconds
- std::chrono::nanoseconds

Пример

std::chrono::seconds sec(60); // 60 секунд

std::chrono::milliseconds ms(500); // 500 миллисекунд

3. **Часы**

В std::chrono существуют различные типы часов, которые позволяют измерять время:

- std::chrono::steady_clock: Часы с монотонным временем, т.е. они всегда идут вперед и не могут быть изменены
- std::chrono::system_clock: Часы, которые синхронизированы с реальным временем (например, время UNIX)
- std::chrono::high_resolution_clock: Часы с самой высокой точностью, доступной в системе

4. **Методы**

- std::chrono::duration_cast<T>

Преобразует одну продолжительность в другую единицу времени.

Пример

std::chrono::milliseconds ms = std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::seconds(5));

- std::chrono::steady_clock::now()

Получение текущего времени по монотонным часам.

Пример

auto start = std::chrono::steady_clock::now();

- std::chrono::system_clock::now()

Получение текущего времени по системным часам.

Пример

auto start = std::chrono::system_clock::now();

- std::this_thread::sleep_for()

Задержка на указанное количество времени.

Пример

std::this_thread::sleep_for(std::chrono::seconds(1)); // Задержка на 1 секунду

- std::this_thread::sleep_until()

Задержка до указанного времени.

Пример

auto future_time = std::chrono::system_clock::now() + std::chrono::seconds(5);

std::this_thread::sleep_until(future_time);

5. **Работа с временем**

Разница между двумя точками времени (например, для измерения времени выполнения какого-либо кода):

auto start = std::chrono::high_resolution_clock::now();

// код, который выполняется

auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Продолжительность: " << duration.count() << " milliseconds" << std::endl;

Пример использования std::chrono для измерения времени

#include <iostream>

#include <chrono>

#include <thread>

int main() {

auto start = std::chrono::steady_clock::now(); // Начало измерения

std::this_thread::sleep_for(std::chrono::seconds(2)); // Задержка 2 секунды

auto end = std::chrono::steady_clock::now(); // Конец измерения

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); // Разница во времени

```
    std::cout << "Elapsed time: " << duration.count() << " milliseconds" << std::endl;

    return 0;

}
```

Этот код измеряет время, которое прошло между вызовами start и end, и выводит продолжительность в миллисекундах.